

Master's Project Work

# Interactive DSM Software

Refactoring of a Direct Stiffness Method Solver and Design of a Graphical User Interface

Maximilian Eichenberger ETH-Nr: 20-928-313

07. January 2026

Prof. Dr. Eleni Chatzi, Dr. A. Egger & F. Betti





Eidgenössische Technische Hochschule Zürich  
Swiss Federal Institute of Technology Zurich

## Declaration of originality

The signed declaration of originality is a component of every semester paper, Bachelor's thesis, Master's thesis and any other degree paper undertaken during the course of studies, including the respective electronic versions.

Lecturers may also require a declaration of originality for other written papers compiled for their courses.

---

I hereby confirm that I am the sole author of the written work here enclosed and that I have compiled it in my own words. Parts excepted are corrections of form and content by the supervisor.

**Title of work** (in block letters):

**Authored by** (in block letters):

*For papers written by groups the names of all authors are required.*

**Name(s):**

**First name(s):**

|  |  |
|--|--|
|  |  |
|  |  |
|  |  |
|  |  |

With my signature I confirm that

- I have committed none of the forms of plagiarism described in the '[Citation etiquette](#)' information sheet.
- I have documented all methods, data and processes truthfully.
- I have not manipulated any data.
- I have mentioned all persons who were significant facilitators of the work.

I am aware that the work may be screened electronically for plagiarism.

**Place, date**

**Signature(s)**

|  |  |
|--|--|
|  |  |
|  |  |
|  |  |
|  |  |

*For papers written by groups the names of all authors are required. Their signatures collectively guarantee the entire content of the written paper.*

## Interactive DSM Software

Maximilian Eichenberger  
20-928-313  
Chair of Structural Mechanics and Monitoring, IBK  
ETH Zürich  
`emaximili@student.ethz.ch`

Supervisors:

Prof. Dr. Eleni Chatzi  
Chair of Structural Mechanics and Monitoring, IBK,  
ETH Zürich  
`chatzi@ibk.baug.ethz.ch`

Dr. Adrian Egger  
Chair of Structural Mechanics and Monitoring, IBK,  
ETH Zürich  
`aegger@ethz.ch`

Federico Betti  
Chair of Structural Mechanics and Monitoring, IBK,  
ETH Zürich  
`federico.betti@ibk.baug.ethz.ch`

07.01.2026

## Abstract

The Direct Stiffness Method (DSM) forms a central part of structural analysis education in Baustatik II at ETH Zürich, yet its algorithmic nature makes the transition from theoretical formulation to practical computation challenging for many students. This project builds upon an existing DSM-based MATLAB solver and focuses on improving its structure, robustness, and usability for educational purposes.

The numerical core was systematically refactored to align with the stepwise DSM workflow taught in the course, improving readability and maintainability while preserving numerical correctness. A layered sanitization and validation concept, together with unit and system tests, was introduced to support reliable future development. In addition, a graphical user interface was implemented that enables intuitive system definition, live visualization, and robust input checking.

The resulting software provides an interactive learning environment and establishes a modular and sustainable foundation for future student projects.

## Keywords

Direct Stiffness Method, Code Refactoring, Educational Software, Graphical User Interface, MATLAB

## Contents

|                                                         |    |
|---------------------------------------------------------|----|
| List of Figures . . . . .                               | 2  |
| 1 Introduction . . . . .                                | 4  |
| 2 Background and Related Work . . . . .                 | 5  |
| 2.1 Direct Stiffness Method in Teaching . . . . .       | 5  |
| 2.2 Previous DSM Student Projects and Tools . . . . .   | 6  |
| 2.3 Identified Gap . . . . .                            | 7  |
| 3 Project Scope and Objectives . . . . .                | 8  |
| 4 Software Architecture and Design . . . . .            | 9  |
| 4.1 Design Philosophy . . . . .                         | 9  |
| 4.2 Overall System Architecture . . . . .               | 9  |
| 4.3 Module Structure . . . . .                          | 11 |
| 4.4 Data Structures and Interfaces . . . . .            | 12 |
| 4.5 Expandability by Design . . . . .                   | 12 |
| 5 Software Implementation and Refactoring . . . . .     | 13 |
| 5.1 Initial Codebase and Identified Issues . . . . .    | 13 |
| 5.2 Refactoring Strategy and Process . . . . .          | 13 |
| 5.3 Performance-Conscious Improvements . . . . .        | 15 |
| 5.4 Sanitization and Validation Layer . . . . .         | 17 |
| 5.5 Testing Strategy . . . . .                          | 18 |
| 6 Graphical User Interface and Intended Usage . . . . . | 20 |
| 6.1 GUI Design Principles . . . . .                     | 20 |
| 6.2 GUI Architecture . . . . .                          | 24 |
| 6.3 User Workflow Overview . . . . .                    | 25 |
| 6.4 Live Visualization . . . . .                        | 25 |
| 7 Limitations and Shortcomings . . . . .                | 27 |
| 8 Future Work and Expansion Potential . . . . .         | 29 |
| 9 Outlook and Conclusion . . . . .                      | 32 |
| 10 References . . . . .                                 | 33 |
| A User Guide: Graphical User Interface . . . . .        | 34 |
| A.1 Getting Started . . . . .                           | 34 |
| A.2 Workflow Summary (Checklist) . . . . .              | 34 |
| A.3 Tab Descriptions . . . . .                          | 34 |

|                                               |    |
|-----------------------------------------------|----|
| A.4 Validation and Error Messages . . . . .   | 36 |
| A.5 Live Viewer Interaction . . . . .         | 36 |
| B Example Results from the Software . . . . . | 37 |

## List of Figures

|                                                                                                 |    |
|-------------------------------------------------------------------------------------------------|----|
| 1 DSM pseudo-code from the lecture . . . . .                                                    | 5  |
| 2 Architecture and pipeline overview of the software . . . . .                                  | 10 |
| 3 High-level folder structure of the DSM software . . . . .                                     | 11 |
| 4 MATLAB Test Browser used for executing unit and system tests . . . . .                        | 19 |
| 5 Overview of the GUI upon startup . . . . .                                                    | 21 |
| 6 Node and beam numbering of an example system . . . . .                                        | 22 |
| 7 Example of icons used in the GUI . . . . .                                                    | 22 |
| 8 Input description example for the cross-section table . . . . .                               | 23 |
| 9 Example units reference tables . . . . .                                                      | 23 |
| 10 Example system and influence line results . . . . .                                          | 37 |
| 11 System definition and live visualization in the graphical user interface . . . . .           | 37 |
| 12 Internal force diagrams (normal force $N$ , shear force $V$ , bending moment $M$ ) . . . . . | 38 |
| 13 Support reactions computed by the DSM solver . . . . .                                       | 38 |

## List of abbreviations

DSM = Direct Stiffness Method

GUI = Graphical User Interface

DOF = Degree of Freedom

FEM = Finite Element Method

ETHZ = Eidgenössische Technische Hochschule Zürich

## Acknowledgement

I would like to thank my supervisors, Dr. Adrian Egger and Federico Betti, for their close guidance throughout the semester. The weekly meetings provided the opportunity to discuss open questions in detail and consistently receive valuable feedback and insights into the software development process as a whole. Adrian Egger's practical expertise in software development, graphical user interfaces, and user experience design was instrumental to this project and further strengthened my knowledge and interest in software development. Federico Betti's feedback and advice based on his coding experience, as well as his introduction to good coding practices and collaborative GitHub workflows were extremely valuable and greatly enhanced my understanding of software development in a team environment.



# 1 Introduction

The Direct Stiffness Method (DSM) is a central component of the bachelor-level course *Baustatik II* at ETH Zürich and forms the basis of most modern structural analysis software. Owing to its matrix-based and algorithmic formulation, the method is particularly well suited for the analysis of statically indeterminate structures. At the same time, this algorithmic nature poses a significant challenge for many students: while the theoretical formulation is introduced rigorously in lectures, the transition from abstract equations to a coherent computational procedure is often difficult to grasp, especially for larger systems or advanced modeling features.

To support this transition, *Baustatik II* employs a range of complementary teaching formats, including lectures, exercises, and MATLAB-based assignments. Nevertheless, developing a consistent mental model of the DSM workflow remains demanding. Interactive educational software can provide valuable support in this context, as it allows students to construct structural systems, modify parameters, and immediately observe the resulting effects on displacements, internal forces, and reactions. Such experimentation helps to reinforce conceptual understanding by directly linking modeling decisions to structural behavior.

The present project builds upon a bachelor thesis by Thurga Kirupakaran, who developed a modular DSM-based MATLAB solver capable of computing internal forces, support reactions, and influence lines for two-dimensional systems (Kirupakaran (2023)). Her work focused on the correctness and extensibility of the numerical DSM core. Interaction with the software was performed exclusively through manually edited input files, and no graphical user interface was provided. As a result, despite its solid technical foundation, the software remained difficult to use in everyday student practice.

This project extends the existing DSM core in two main ways. First, the code is systematically refactored to improve readability, maintainability, and alignment with the DSM procedure as taught in *Baustatik II*. In addition, a comprehensive set of unit and system tests is introduced to ensure correctness and support future development. Second, a graphical user interface is developed that enables intuitive and interactive definition of systems, loads, and output quantities. The GUI is designed with a strong focus on student usability, robust input validation, and a clear modeling workflow.

The overarching goal of this work is to create a unified, modular and student-friendly DSM software that bridges the gap between theoretical formulation, computational implementation, and exploratory learning. This report documents the conceptual and technical aspects of the project, including the software architecture, refactoring strategy, and graphical interface design. Limitations of the current implementation and directions for future extensions are also discussed with the aim of providing a sustainable foundation for subsequent student projects within the context of *Baustatik II*.

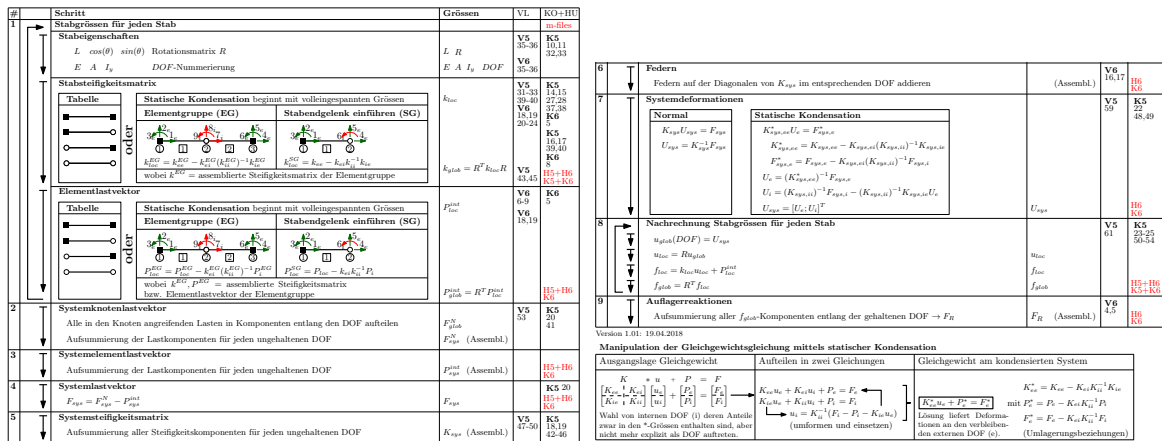
## 2 Background and Related Work

### 2.1 Direct Stiffness Method in Teaching

The Direct Stiffness Method is a displacement-based approach for the analysis of determinate and indeterminate structures and forms a central part of the *Baustatik II* course at ETH Zürich. In contrast to force-based methods, equilibrium is formulated at the system level in terms of nodal displacements, leading to a matrix-based solution procedure that is well suited for computer implementation. For this reason, DSM serves as a natural entry point for introducing students to computational structural mechanics.

In *Baustatik II*, DSM is taught as a clearly structured sequence of computational steps, ranging from the definition of element properties and degrees of freedom to system assembly, application of boundary conditions, solution of the linear system, and post-processing of internal forces and reactions. This stepwise workflow is explicitly emphasized in lecture material and handouts, where it is summarized in pseudo-code form to highlight the individual operations and their order (see Fig. 1).

Figure 1: DSM pseudo-code form the lecture



From a didactic perspective, DSM is primarily taught as a hand-based method, ensuring that students understand the numerical structure of the algorithm. While this approach is effective for building conceptual understanding, it limits the size and complexity of systems that can be treated. An interactive software environment that preserves the stepwise logic of the hand-based DSM approach while enabling the analysis of larger systems can provide additional educational value.



## 2.2 Previous DSM Student Projects and Tools

Over the past years, several student projects at ETH Zürich have explored the development of educational software for structural analysis, particularly in the context of *Baustatik II*. These projects differ considerably in their methodological focus, degree of interactivity, and didactic objectives, and together illustrate a wide range of approaches to supporting students in learning the Direct Stiffness Method.

A first group of projects places a strong emphasis on didactic guidance and theoretical transparency. Notable examples are the works by Ana da Silva Rodrigues and Tinet Ruch, which focus on teaching displacement- and force-based methods through guided, step-by-step workflows. Their implementations closely follow the lecture material and classical hand-based solution procedures, providing extensive explanations, intermediate results, and conceptual support. While highly pedagogical, these tools deliberately prioritise guided understanding and transparency of the underlying methods over general-purpose automated analysis.

A second line of work emphasizes interactive modeling and extended numerical functionality. The MATLAB-based software developed by (Bundi and Ferrari (2018)) is a representative example. It offers a comprehensive graphical interface for defining frame structures, loads, and cross-sections, along with extensive visualization capabilities. The software supports elastic and plastic analysis and includes additional tools such as cross-section property calculators. However, its numerical core is based on the Finite Element Method rather than the Direct Stiffness Method, and its primary focus lies on user interaction and visual feedback rather than on explicitly illustrating the DSM workflow as taught in *Baustatik II*.

An application developed by Salathé and Beeler (2020) aligns more closely with the DSM formulation used in the course. Their software explicitly displays intermediate calculation steps and allows students to follow the numerical procedure in detail, thereby strengthening the connection between theoretical formulation and computational implementation.

The direct predecessor of the present project is the bachelor thesis by (Kirupakaran (2023)). Her work implemented a robust and extensible DSM-based solver capable of computing internal forces, support reactions, and influence lines for two-dimensional systems. The project covers the full DSM pipeline and supports a wide range of modeling features, including different support types, springs, nodal and member loads, and prescribed displacements. However, interaction with the software relied exclusively on manually edited input files and did not include a graphical user interface. As a result, despite its strong numerical foundation, accessibility for everyday student use and long-term maintainability remained limited.

Taken together, these projects demonstrate a variety of successful but specialized approaches to educational DSM software, each addressing specific teaching objectives while leaving room for further integration of didactic clarity, interactivity, and extensibility.

## 2.3 Identified Gap

The review of previous DSM-based educational software shows that many challenges in teaching structural analysis are already well understood and have been addressed successfully in isolation. Some projects provide strong didactic support and theoretical transparency, others focus on advanced numerical capabilities, and some emphasize intuitive modeling and visual feedback. However, these strengths are rarely combined within a single, coherent software environment.

From an educational perspective, a gap remains between hand-based DSM teaching and the use of general-purpose solvers. Manual calculations lead to a deep understanding of individual DSM steps but do not scale to larger systems. Script-based computational tools extend problem size but often reduce interactivity and make it harder to relate modeling changes directly to structural behavior. An interactive environment that preserves the stepwise logic of the Direct Stiffness Method while enabling the analysis of larger systems therefore has clear educational potential.

This project deliberately focuses on establishing a robust and extensible foundation for DSM-based educational software. Emphasis is placed on a clearly structured computational core, explicit separation of concerns between computation and interaction, comprehensive input validation, and systematic testing to support long-term maintainability. By prioritizing modularity and correctness over feature completeness, the resulting framework is intended to serve as a stable basis that can be extended with additional pedagogical features, analysis capabilities, or visualizations in future work without compromising clarity or reliability. These considerations directly informed the design goals and scope of the present project.

### 3 Project Scope and Objectives

This project does not aim to redesign the numerical method itself. Instead, it focuses on restructuring the existing implementation to improve clarity, modularity, and accessibility, and on extending the software with a graphical user interface suitable for everyday student use.

The scope of the project is defined by the following primary objectives:

#### Objectives

- Refactoring the existing DSM implementation to improve readability, modularity, and alignment with the stepwise DSM procedure taught in *Baustatik II*, including a clear separation between input handling, numerical computation, and post-processing.
- Establishing a robust validation and testing framework, comprising unit tests and selected system-level tests, to ensure correctness and support safe future development.
- Developing an extensible and user-friendly graphical user interface that enables intuitive system definition, robust input validation, and interactive use.

**Non-Goals** Several aspects are explicitly outside the scope of this project:

- Redesign or extension of the underlying DSM formulation or influence line algorithms beyond minor structural adjustments.
- Comprehensive redesign of output visualization; only targeted improvements such as consistent scaling and clearer labeling are implemented.
- Advanced analysis features such as nonlinear material behavior, geometric nonlinearity, dynamic analysis, or plastic collapse modeling.

## 4 Software Architecture and Design

### 4.1 Design Philosophy

The design of the software aims to balance robustness, clarity, and didactic accessibility. As the tool is intended primarily for educational use, transparency and readability are prioritized over maximal computational performance. In particular, the software is designed to remain accessible to students with little or no prior programming experience.

This philosophy is reflected in two central architectural decisions. First, the implementation deliberately avoids a fully object-oriented design. Instead, the code follows a functional structure based on clearly defined functions and structured data containers. This keeps the execution flow explicit and traceable, allowing students to follow the DSM computation step by step without requiring familiarity with advanced object-oriented concepts.

Second, the main analysis routine (`DirectStiffnessMethod.m`) is structured to mirror the stepwise DSM procedure taught in *Baustatik II*. Each conceptual step of the method is represented explicitly and executed in the same logical order as in the lecture pseudo-code (Fig. 1). This alignment enables a direct correspondence between the theoretical workflow presented in the course and its concrete implementation in the software.

During refactoring, the originally monolithic implementation of the main DSM function was decomposed into smaller, task-specific functions. Each function performs a single well-defined operation, such as assembling system matrices, applying constraints, or computing internal forces. This improves readability, reduces code duplication, and enables systematic unit testing. As a result, the main DSM routine now primarily acts as an orchestrator that coordinates the individual stages of the analysis and handles branching logic for different analysis modes (e.g. influence lines or static condensation).

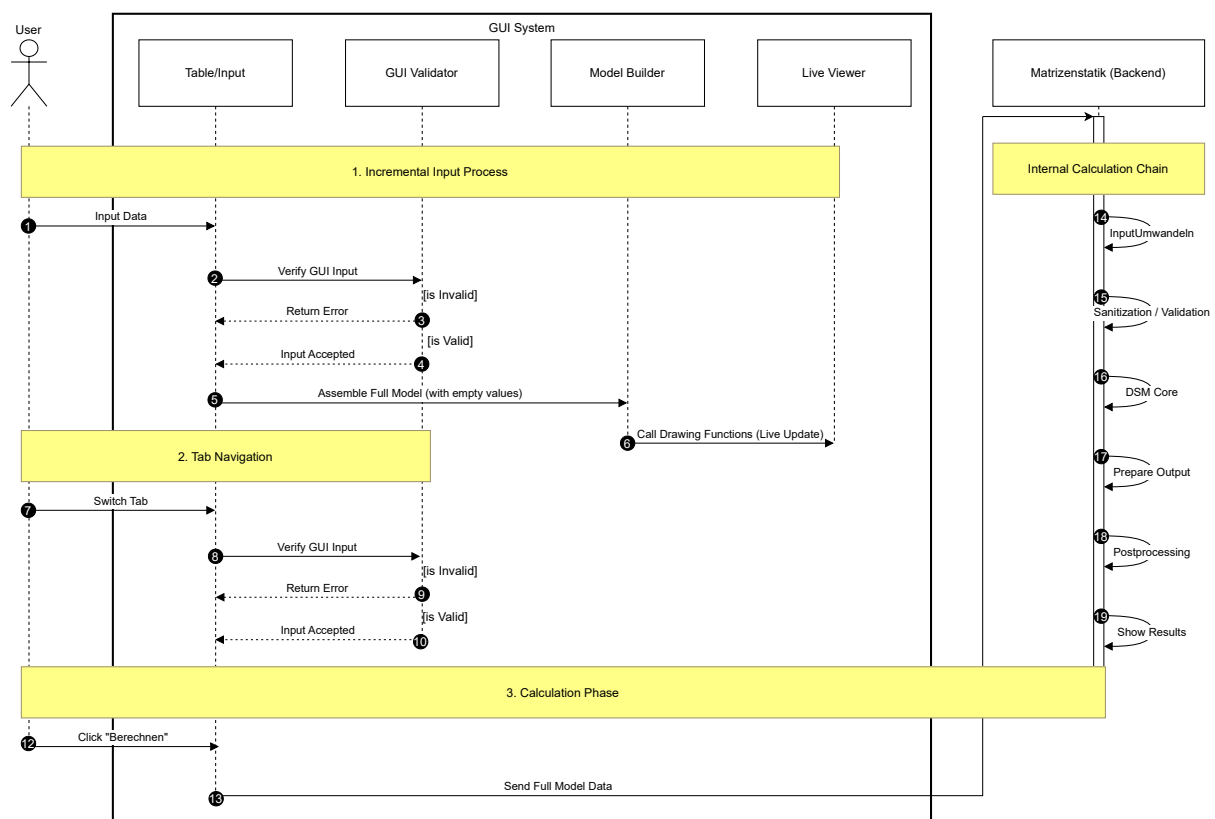
### 4.2 Overall System Architecture

The system architecture separates GUI-driven interaction from the backend analysis pipeline orchestrated by `MatrizenStatik`. A schematic overview of the GUI and backend structure, including calling order and validation stages, is shown in Fig. 2.

The left part of the diagram illustrates the incremental GUI workflow, where input is validated, assembled into a temporary model, and visualized during system definition without triggering numerical computation. The backend pipeline on the right-hand side is activated only when the user initiates the calculation. At this point, the complete model is transferred to the solver and processed through the following stages:

1. conversion of GUI data into a solver-compatible input structure,
2. input sanitization and validation,
3. execution of the DSM core,
4. preparation of results for visualization.

Figure 2: Architecture and pipeline overview of the software

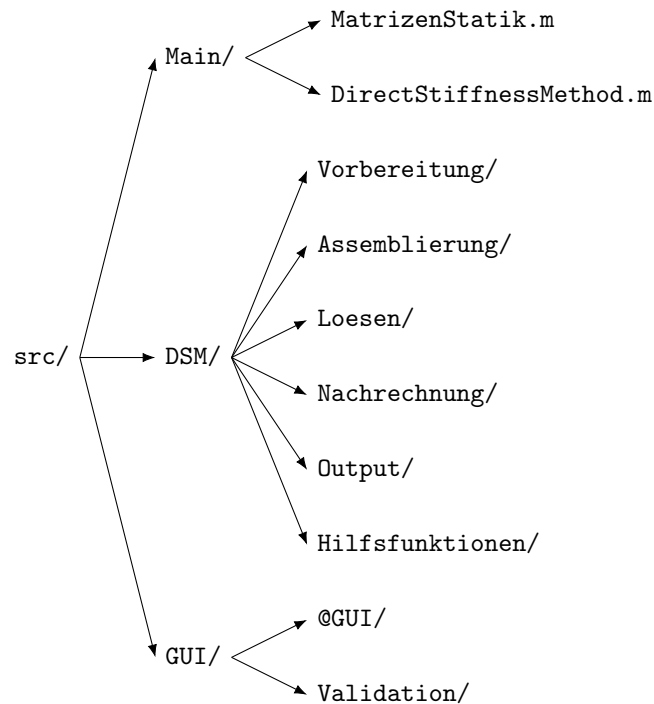


Compared to the original implementation, the fundamental solver pipeline remains unchanged. The main architectural addition introduced during refactoring is a dedicated validation and sanitization layer, which checks input consistency and guards against common modeling errors before numerical routines are executed.

User interaction with the software is entirely GUI-driven. During system construction, partial input is validated incrementally and used to generate a temporary preview model for live visualization. Only when the user explicitly triggers the computation is the complete input passed to the backend, where full validation is performed prior to executing the solver.

To manage the growing codebase, functions are organized into folders according to responsibility and calling order. An overview of the folder structure is shown in Fig. 3.

Figure 3: High-level folder structure of the DSM software



### 4.3 Module Structure

The backend is organised into function groups corresponding to the logical stages of the DSM workflow. Table 1 summarises the responsibilities of the main folders and their role within the analysis pipeline.

Table 1: Overview of backend modules and responsibilities

| Folder          | Responsibility                                                     |
|-----------------|--------------------------------------------------------------------|
| Vorbereitung    | Input sanitisation, validation, and conversion to solver format    |
| Assemblierung   | DOF numbering, element creation, global matrix and vector assembly |
| Loesen          | Static condensation and solution of linear systems                 |
| Nachrechnung    | Computation of reactions and internal forces                       |
| Output          | Plotting and preparation of results for GUI display                |
| Hilfsfunktionen | Reusable low-level utilities (e.g. transformations, releases)      |

This organization ensures a clear separation of concerns without introducing additional abstraction layers. Each module corresponds to a distinct stage of the DSM procedure, and the calling order of functions closely follows the conceptual workflow taught in the course.



## 4.4 Data Structures and Interfaces

All core data is stored in a single overarching MATLAB structure **Model**, which is subdivided into three main parts: **Input**, **Analyse**, and **Output**. This separation distinguishes user-defined data, intermediate computational quantities, and result data.

The **Input** structure contains the complete system definition and is created via the GUI using **guiToInput**, which normalizes table-based UI data into a solver-compatible format. This isolates GUI-specific data handling from the numerical core.

The **Analyse** structure is populated during execution of the DSM solver and contains geometric data, system matrices, and solution vectors. For static condensation, reduced matrices and vectors are grouped explicitly in a dedicated substructure. The **Output** structure stores all quantities required for result presentation and visualization.

Overall, this organization enforces well-defined interfaces between modules, simplifies testing, and reduces the risk of unintended side effects when extending or modifying the codebase.

## 4.5 Expandability by Design

The software architecture was deliberately designed to support future extensions with minimal restructuring. Due to the function-oriented backend and clearly defined interfaces, new functionality can be integrated by adding or extending functions at appropriate points in the existing pipeline without affecting unrelated components.

Potential future extensions of the software are discussed in detail in Section 8.

The graphical user interface follows a panel-based structure that naturally supports extension. Individual input categories are implemented as separate panels, allowing new tabs or expanded tables to be added without redesigning the overall layout. For example, material and cross-section definitions can be extended with additional properties, or new analysis-selection panels can be introduced independently of existing result visualization.

Overall, the combination of a structured functional backend and an expandable GUI provides a flexible foundation for future student projects.

## 5 Software Implementation and Refactoring

### 5.1 Initial Codebase and Identified Issues

A central issue in the initial codebase was the concentration of many logically distinct DSM steps into a single large routine. In the original structure, the file `DSM/DirectStiffnessMethod.m` contained most of the analysis pipeline in one place: element property handling, load vector construction, global assembly, application of boundary conditions, static condensation, solution of the system, and post-processing. While this organisation made the overall workflow readable at a high level, it limited maintainability and extensibility, as modifications in one part of the function could easily affect unrelated steps.

In the refactored codebase, the DSM workflow is preserved, but the main routine is reduced to an orchestrator that mirrors the lecture pseudo-code and delegates responsibilities to dedicated functions. As a measurable indicator of this restructuring, the size of `DirectStiffnessMethod.m` was reduced from approximately 429 lines to 61 lines, reflecting the transition from a monolithic implementation to a clearly structured pipeline of specialized functions.

The original system was designed for input files, where all parameters were defined explicitly and consistently before execution. Although a basic input check existed in the form of `istGueltigerInput.m`, much of the solver logic relied on implicit assumptions regarding input completeness and consistency. This becomes limiting in an interactive, GUI-driven workflow, where input is inherently incomplete and evolves during system construction.

In the refactored version, input is therefore checked and validated at multiple stages, depending on context:

1. lightweight validation during model construction to support live visualization,
2. page-level validation when switching between GUI tabs,
3. strict validation prior to numerical computation to ensure solver correctness.

An overview of these validation stages is shown in Fig. 2.

### 5.2 Refactoring Strategy and Process

The primary goals guiding the refactoring process were to establish a clear separation of concerns and to restructure `DirectStiffnessMethod` so that its execution order matches the stepwise DSM pseudo-code presented in the course. The intention was not to alter the numerical procedure, but to make the implementation structure reflect the conceptual algorithm taught in class.

To achieve this, the originally monolithic solver routine was decomposed into functional blocks, each responsible for a single task within the analysis pipeline. These blocks were factored into separate functions with well-defined inputs and outputs, and care was taken to ensure that each function received only the data required for its responsibility. Repeated logic was extracted into shared helper functions to reduce duplication and improve consistency.

The refactoring also clarified the responsibility boundaries between major parts of the solver (input preparation and validation, system assembly, solution procedures, post-processing, and output preparation). This makes dependencies explicit and reduces the likelihood that changes in one part of the pipeline affect unrelated steps.

General structural improvements undertaken during the refactoring include:

- reduction of code repetition,
- simplification of conditional logic,
- selective vectorization of loops,
- consistent use of sparse matrices for system assembly,
- addition of clear, German-language comments to support student comprehension.

Performance-oriented changes such as vectorization and sparse matrix assembly were applied selectively and only where they did not compromise code readability or didactic clarity.

During early stages of refactoring, correctness was ensured by preserving the original calculation order and repeatedly verifying results against existing example input files. In later stages, once unit and system tests were introduced, these tests were executed after each major change and prior to committing new code. This reduced the risk of unintended regressions and provided a clear reference during code review.

In the final stages of the project, additional improvements were made to the visual robustness of the output, particularly for GUI-based result display. These included normalizing the visual magnitude of force arrows, attaching labels to arrows to improve readability during zooming, and consistently displaying force magnitudes as absolute values while conveying direction through arrow orientation. Although these changes do not affect numerical results, they improve stability and readability of visual output.

During the refactoring process, the MATLAB codebase was additionally formatted using the tool *MBeautifier* (Varga (2024)). This automatic formatter was used to enforce consistent indentation, spacing, and line breaking across the project.

Changes related to the graphical user interface are discussed separately in Section 6.

### 5.3 Performance-Conscious Improvements

While performance optimization was not a primary objective, several refactoring steps naturally improved efficiency and scalability without obscuring the DSM logic. In particular, MATLAB often benefits from vectorized operations and sparse assembly patterns, which can often be applied while keeping the implementation transparent.

**Vectorization of for-loops** A representative example is the assembly of nodal loads into the global force vector. In the original implementation, nodal loads were processed sequentially in a loop:

Listing 1: Original loop-based nodal load assembly

---

```
for i = 1:Info.nKnotenLasten
    if KnotenLast(i).DOF ~= 0
        F_sys_Knoten(KnotenLast(i).DOF) = ...
            F_sys_Knoten(KnotenLast(i).DOF) + KnotenLast(i).val;
    end
end
```

This approach mixes three tasks in one place: computing global indices, checking admissibility, and updating the force vector. In the refactored implementation, the same operation is expressed in vectorized form by computing indices in bulk, filtering invalid entries with logical masks, and assembling the result using MATLAB's sparse constructor:

Listing 2: Vectorised nodal load assembly using sparse indexing

---

```
KL = model.KnotenLast(:);
node = [KL.node]';
dir = [KL.dir]';
val = [KL.val]';

local = (node - 1) * nkd + dir;
ok = local >= 1 & local <= numel(DOF);
g = zeros(size(local));
g(ok) = DOF(local(ok));
keep = g > 0;

F_N = sparse(g(keep), 1, val(keep), nDOF, 1);
```

This formulation makes the separation between index computation, validation, and assembly explicit. Similar vectorization strategies were applied in other assembly routines.

**Sparse matrix assembly** A second important improvement concerns the assembly of the global stiffness matrix. In DSM, the global stiffness matrix is typically large but sparse, since each element only contributes to a small subset of global degrees of freedom. The original implementation allocated the stiffness matrix densely and assembled it by repeatedly updating sub-blocks:

Listing 3: Original dense stiffness matrix assembly

---

```
K_sys = zeros(nDOF, nDOF);

for i = find(~[Stab.inTeilSys])
    K_sys(Stab(i).DOF, Stab(i).DOF) = ...
        K_sys(Stab(i).DOF, Stab(i).DOF) + ...
        Stab(i).k_glob( ...
            Stab(i).activeStabDOF, ...
            Stab(i).activeStabDOF );
end
```

While conceptually close to the mathematical formulation, this approach repeatedly modifies a large dense matrix, which becomes inefficient in memory usage and execution time as system size increases. In the refactored implementation, stiffness contributions are collected as index-value triplets and assembled in a single call to MATLAB's sparse constructor:

Listing 4: Sparse global stiffness matrix assembly using index triplets

---

```
Icell = cell(numel(Stab), 1);
Jcell = cell(numel(Stab), 1);
Vcell = cell(numel(Stab), 1);

% element loop: compute row indices, column indices and values and store them in Icell{i},
%               Jcell{i}, Vcell{i}

I = vertcat(Icell{1:wptr});
J = vertcat(Jcell{1:wptr});
V = vertcat(Vcell{1:wptr});

K_sys = sparse(I, J, V, nDOF, nDOF);
```

This strategy reflects sparsity explicitly and avoids repeated dense matrix updates. The resulting stiffness matrix is sparse from the outset and can be passed directly to MATLAB's sparse linear solvers. Importantly, the sparse formulation remains transparent and closely aligned with the theoretical DSM structure.

## 5.4 Sanitization and Validation Layer

The sanitization and validation layers play a central role in ensuring robustness, consistency, and usability of the software. Their purpose is to guarantee that only well-defined input reaches the numerical solver, while providing meaningful feedback that helps users identify and correct modelling errors.

The *sanitization* step ensures structural completeness of the input by creating missing optional fields and default values. This allows solver routines to operate on consistent data structures without repeatedly checking for field existence. A representative example is the sanitization of nodal load definitions:

Listing 5: Example sanitization step for nodal loads

---

```
for i = 1:numel(analysis.KnotenLast)
    if ~isfield(analysis.KnotenLast(i), 'node'), analysis.KnotenLast(i).node = NaN; end
    if ~isfield(analysis.KnotenLast(i), 'dir'), analysis.KnotenLast(i).dir = NaN; end
    if ~isfield(analysis.KnotenLast(i), 'val'), analysis.KnotenLast(i).val = NaN; end
end
```

In contrast, the *validation* step checks admissibility and physical consistency, including completeness, data types, ranges, and logical relationships between model components. Invalid input is not silently corrected; instead, descriptive error messages are generated. An example validation routine for nodal definitions is shown below:

Listing 6: Example validation step for node definitions

---

```
nK = numel(A.Knoten);
if numel(A.Stab) > 0 && nK < 2
    issues{end+1} = 'ERROR: Mindestens zwei Knoten erforderlich, wenn Stbe vorhanden sind.';
end
for i = 1:nK
    if ~isfield(A.Knoten(i), 'x') || ~isfield(A.Knoten(i), 'y') ...
        || ~isscalar(A.Knoten(i).x) || ~isscalar(A.Knoten(i).y) ...
        || ~isfinite(A.Knoten(i).x) || ~isfinite(A.Knoten(i).y)
        issues{end+1} = sprintf('ERROR: Knoten %d hat unglte Koordinaten.', i);
    end
end
```

The validation logic was designed to cover relevant edge cases so that the numerical core can assume admissible input and does not need defensive checks throughout the solver.

With the introduction of the GUI, additional validation became necessary at the interface level, since users frequently create temporarily incomplete models during editing. Lightweight, table-specific validation routines were therefore implemented directly on GUI input data to safely

support live preview updates and to provide feedback in terms that match what users see in the interface. The backend validation remains the final safeguard before numerical computation. This layered approach improves robustness and supports future extensions, since new features can integrate validation logic without compromising overall stability.

## 5.5 Testing Strategy

Unit and system tests formed a major component of the refactoring process. Their purpose is to verify correctness, improve robustness, and enable safe future development. Unit tests target small, well-defined parts of the solver and verify behaviour under controlled input scenarios. This makes it possible to localise errors precisely when future changes are introduced.

All tests were implemented using MATLAB's unit testing framework (`matlab.unittest`). Test classes were designed to target either a single function or a small group of tightly related functions, ensuring that failures remain interpretable while still providing broad coverage.

As an illustrative example, Listing 7 shows a unit test for the function `elementeErzeugen.m`. The test constructs a minimal system consisting of two nodes connected by a single horizontal beam. After executing the function, the output is compared against analytically known reference values (geometry, stiffness matrices, and rotation matrix consistency).

Listing 7: Example unit test verifying geometry and stiffness of a horizontal beam

---

```
function horizontal_bar_geometry_and_stiffness(tc)
    M = tests.util.ModelBuilder.minimal(2, 3);
    S = tests.util.ModelBuilder.oneBarHoriz(M, 1, 2);
    C = tests.util.Const;
    S.Iy = C.I;
    M.Stab = S;
    M.Info.nStaebe = 1;

    M2 = elementeErzeugen(M);

    % geometry
    tc.verifyEqual(M2.Stab(1).L, 1, 'AbsTol', 1e-12);
    tc.verifyEqual(M2.Stab(1).cs, 1, 'AbsTol', 1e-12);
    tc.verifyEqual(M2.Stab(1).sn, 0, 'AbsTol', 1e-12);
    tc.verifyEqual(M2.Stab(1).phi, 0, 'AbsTol', 1e-12);

    % stiffness matrices
    tc.verifySize(M2.Stab(1).k_loc_v, [6, 6]);
    tc.verifySize(M2.Stab(1).k_loc, [6, 6]);
    tc.verifyTrue(issymmetric(M2.Stab(1).k_loc_v));

    % rotation matrix
    R = getR(1, 0);
```



```

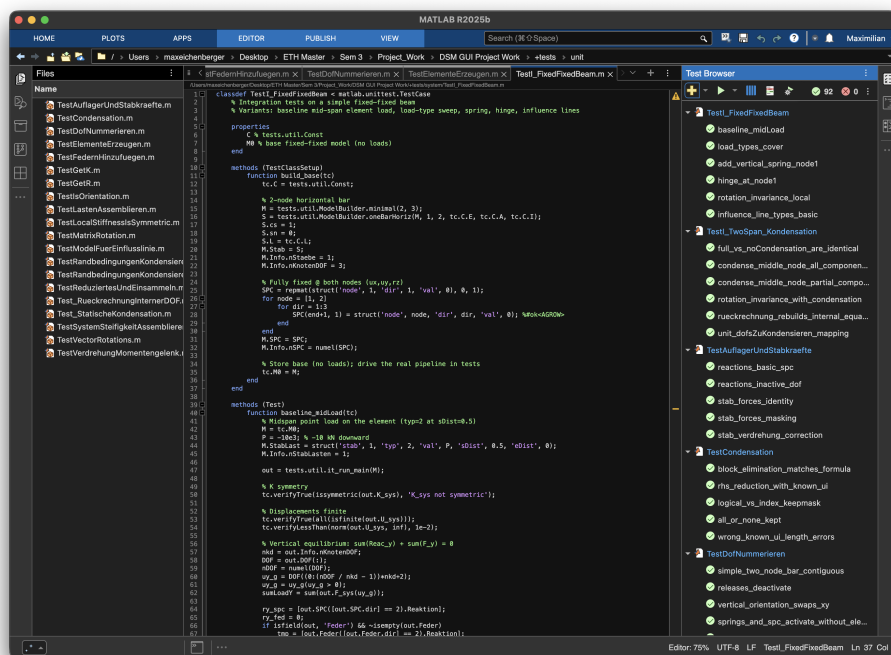
    tc.verifyEqual(M2.Stab(1).R, R, 'AbsTol', 1e-12);
end

```

In addition to unit tests, system tests were implemented to verify the interaction of multiple components across the full analysis pipeline. Two main system tests were created. The first analyses a single-span beam fixed at both ends and verifies reactions, internal forces, and influence lines under various loading, support, hinge, and spring configurations. The second test considers a two-span girder and additionally verifies the correctness of the static condensation implementation.

MATLAB's Test Browser was used during development for running tests selectively and inspecting failures with detailed diagnostics (Fig. 4).

Figure 4: MATLAB Test Browser used for executing unit and system tests



Together, the unit and system tests cover approximately 96% of the executable lines in the DSM core. Output and visualization routines were deliberately excluded from systematic testing, as refactoring and extending the output layer was not a primary objective of this project. The resulting test suite provides a robust safety net for future development and significantly improves confidence in the correctness and stability of the refactored code.

## 6 Graphical User Interface and Intended Usage

### 6.1 GUI Design Principles

The graphical user interface was implemented using MATLAB App Designer (mat (2025)), which provides an integrated environment for developing interactive applications. App Designer separates GUI development into two complementary parts:

1. **Design View**, where visual components such as tables, buttons, panels, and drop-down menus are arranged, and
2. **Code View**, where application logic and interaction behavior are implemented.

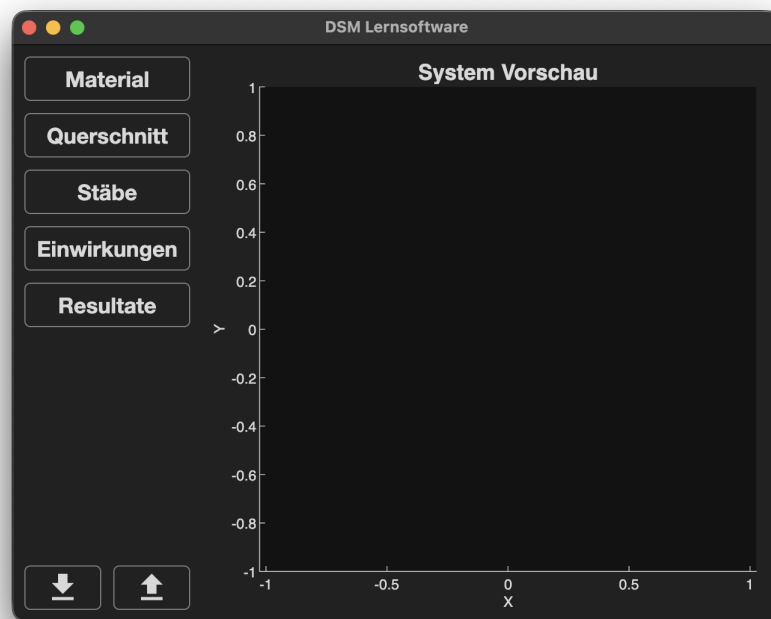
This framework supports structured GUIs with event-driven callbacks, private properties, and helper functions.

The GUI was designed with a strong focus on simplicity and didactic clarity, as the software is intended primarily as a learning tool. The main objective was to allow users to begin modeling structural systems immediately, without requiring extensive prior instruction or external documentation. To achieve this, input operations were designed to require as few interactions as possible and follow a clear, predictable workflow.

User input was intentionally kept close to the formulation used in the lecture and in hand calculations. For example, users must explicitly distinguish between nodal loads and member loads, reflecting the modeling decisions required when applying the Direct Stiffness Method manually.

The GUI layout follows a top-to-bottom modeling workflow using sequential tabs, from system definition to result evaluation. This supports progressive disclosure: only inputs relevant to the current modeling step are visible, preventing the interface from becoming overwhelming. An overview of the GUI upon startup is shown in Fig. 5.

Figure 5: Overview of the GUI upon startup

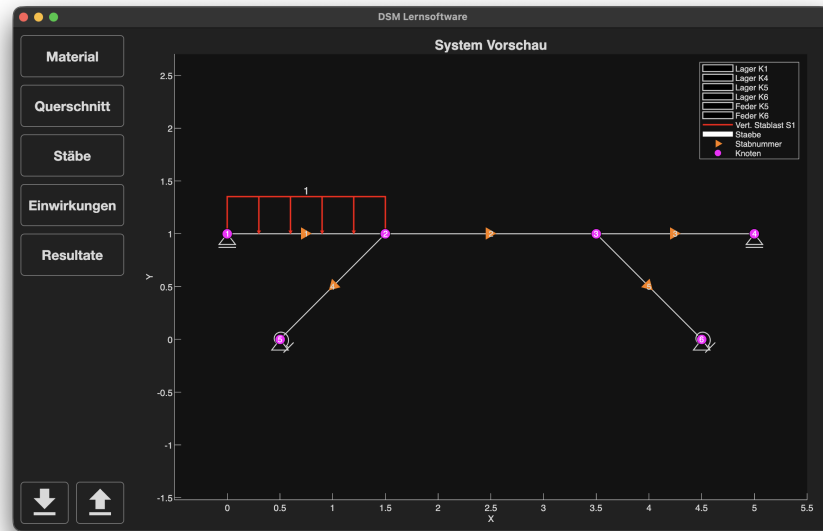


Input fields were grouped into logical sections: materials and cross-sections, geometric definition of nodes and elements (including hinges, supports, and springs), followed by load definitions in a dedicated actions tab. The chosen tab structure is a compromise between simplicity and future expandability, allowing new functionality to be integrated into the appropriate section without a redesign (see Section 4.5).

To reduce invalid input, discrete choices are implemented using drop-down menus wherever possible. Newly defined entities (e.g. materials and nodes) become immediately available in dependent tables (e.g. cross-sections and beams), minimizing manual bookkeeping and ensuring internal consistency.

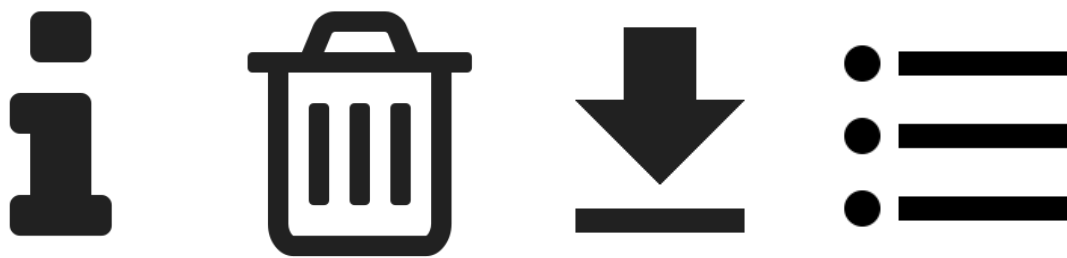
A live visualization component displays the currently defined system, including node and element numbering and element orientation. Visibility of object types and labels can be toggled via the legend. This immediate feedback reinforces the connection between numerical input and the resulting structural model and helps identify modeling errors early.

Figure 6: Node and beam numbering of an example system



The interface was deliberately kept visually uncluttered by limiting complex nested menus and multi-window layouts. Icons were used where appropriate to reduce visual load and provide a consistent visual language (Fig. 7). The icons were obtained using the *MATLAB Icons* App by Pavel Roslovets (Roslovets (2025)). In addition, tooltip hints were added to most buttons to provide short explanations on hover.

Figure 7: Example of icons used in the GUI



In addition to the main input panels, the GUI provides a dedicated information pop-up window accessible via info buttons above the tables. This window contains context-specific explanations of input fields (Fig. 8) and a units overview. Since the solver is unit-agnostic, two standard unit systems are provided with reference tables to support consistent input and reduce unit-related modeling errors (Fig. 9).

Figure 8: Input description example for the cross-section table

The 'Info Fenster' dialog box contains the following input fields and descriptions:

- Name:** Bezeichnung des Querschnitts.
- Material:** Welches Material dem Querschnitt zugeordnet ist.
- Fläche:** Querschnittsfläche A des Stabes. Sie wird für die Berechnung der Normalkraftsteifigkeit EA verwendet.
- Trägheitsmoment:** Flächenträgheitsmoment I des Querschnitts um die relevante Biegeachse. Es wird für die Berechnung der Biegesteifigkeit EI verwendet.
- Dehnstarr:** Option, um den Stab in Dehnrichtung praktisch unverschieblich zu modellieren. Entspricht einem sehr grossen A (dehnstarrer Stab).
- Biegesteif:** Option, um den Stab in Biegung praktisch unverschieblich zu modellieren. Entspricht einem sehr grossen I (biegestarrer Stab).

Figure 9: Example units reference tables

The 'Info Fenster' dialog box displays two unit reference tables. Both tables include a warning: "Die Software führt keine Einheitenumrechnung durch. Bitte verwenden Sie ein konsistentes Einheitensystem für alle Eingaben."

**Left Table (Ingenieur: mm, kN, GPa):**

| E                        | A               | I               | Länge | Kraft | Moment | Linienlast | Federsteifigk... |
|--------------------------|-----------------|-----------------|-------|-------|--------|------------|------------------|
| kN/mm <sup>2</sup> (GPa) | mm <sup>2</sup> | mm <sup>4</sup> | mm    | kN    | kNmm   | kN/mm      | kN/mm            |

**Right Table (SI: m, N, Pa):**

| E                     | A              | I              | Länge | Kraft | Moment | Linienlast | Federsteifigk... |
|-----------------------|----------------|----------------|-------|-------|--------|------------|------------------|
| N/m <sup>2</sup> (Pa) | m <sup>2</sup> | m <sup>4</sup> | m     | N     | Nm     | N/m        | N/m              |

To ensure robustness, the GUI performs systematic input validation at multiple stages: table-level validation before visualization, tab-level validation when switching to the next tab, and strict validation before triggering the numerical solver. If invalid or incomplete input is detected, an error dialog informs the user and prevents inconsistent data from propagating.

Finally, import and export of complete example systems is supported via MATLAB `.mat` files. This allows users to share systems, explore predefined examples, and resume work without re-entering data.

## 6.2 GUI Architecture

The graphical user interface was designed as a strictly separate interaction layer that does not influence the numerical behavior of the structural solver. All computations related to matrix assembly, system solution, and result evaluation are performed exclusively within the DSM solver pipeline. The GUI is responsible only for user interaction, input validation, and visualization.

This separation ensures that the solver can be executed independently of the GUI (e.g. in unit tests) and that numerical correctness does not depend on the graphical implementation. The interface between GUI and solver is defined by a structured input representation assembled within the GUI and passed to the solver in a single step.

This design principle is illustrated by the callback associated with the Calculate button. When pressed, the GUI state is converted into a solver-compatible input structure, validated, passed unchanged to the solver, and the results are subsequently visualized:

Listing 8: Callback when the calculate button is pressed

---

```
function Button_CalculatePushed(app, event)

    % Convert GUI state to solver input
    [in, inputIssues] = guiToInput(app, true);

    if ~inputIssues.ok
        msg = strjoin(string(inputIssues.messages), newline);
        uialert(app.UIFigure, msg, 'Eingabefehler');
        return;
    end

    try
        opts = struct('throwOnError', true);
        [app.Model, status, Meldung, issues] = MatrizenStatik(in, opts);

        disp('Run completed successfully');

    catch ME
        uialert(app.UIFigure, ME.message, 'Error');
    end
end
```

The GUI follows an event-driven programming model: button presses, table edits, and tab changes trigger callbacks. To keep the `.mlapp` maintainable, callbacks were intentionally kept short and delegate most logic to helper functions. Repetitive interactions are handled using generic mechanisms such as tag-based mappings. For example, menu buttons and corresponding panels share the same key, allowing a single callback to switch panels without extensive conditional logic (Listing 9).

Listing 9: Example of a tag-based panel switching callback

---

```
function Button_LeftMenuPushed(app, event)
% get tag of the pressed button
newKey = string(event.Source.Tag);

% If the button belongs to the current visible panel -> do nothing
if ~isempty(app.CurrentPanel) && app.CurrentPanel == app.Panels.(newKey)
    return;
end

% make the old panel invisible
if ~isempty(app.CurrentPanel) && isvalid(app.CurrentPanel)
    app.CurrentPanel.Visible = 'off';
end

app.Panel_SystemViewer.Layout.Column = [4 5];
newPanel = app.Panels.(newKey);
newPanel.Visible = 'on';
app.CurrentPanel = newPanel;

end
```

### 6.3 User Workflow Overview

The intended workflow is sequential and follows the tab structure shown in Fig. 5. A complete user guide is provided in Appendix A. In short, using the GUI consists of the following steps:

1. Define materials and cross-sections.
2. Define nodes and beams (including hinges), then assign supports and optional springs.
3. Define actions (nodal loads, member loads, prescribed displacements).
4. Select analysis outputs and compute results (internal forces, reactions, influence lines; optional static condensation).
5. Inspect results and, where relevant, in the MATLAB workspace (e.g. condensed matrices).

### 6.4 Live Visualization

The live viewer provides continuous visual feedback during system construction by displaying nodes, elements, supports, and applied actions as soon as they are fully defined. This enhances usability and supports conceptual understanding by linking numerical input directly to the resulting structural model.

A key implementation challenge is robustness against incomplete or inconsistent input during

interactive editing. To ensure stability, visualization updates are protected by a validation guard: only complete and admissible entities are visualized. Instead of drawing raw table entries, a temporary *preview model* is assembled when updates are triggered and is passed to the drawing routines only if validation checks are satisfied. This prevents runtime errors and ensures that the viewer always represents a coherent intermediate system state.

Dynamic input changes (e.g. deleting nodes or beams that are referenced elsewhere) are handled through dependency checks and update rules that remove or invalidate affected entries as needed. Visualization is restricted to valid rows, so inconsistent intermediate states are never rendered.

Finally, the visualization logic was refined to improve scaling and legibility. Forces and moments are displayed with consistent relative magnitudes independent of the absolute system size, and magnitude labels are positioned and scaled so they remain readable across different configurations. Together, these measures transform the GUI into a robust interactive modeling environment rather than a purely table-driven input form.



## 7 Limitations and Shortcomings

Although the project achieved its primary objectives – namely improving maintainability and conceptual clarity of the DSM core while providing a student-friendly graphical interface – several limitations remain. Some are inherent trade-offs made deliberately in favor of didactic transparency, while others represent concrete areas that could be improved in future work.

**Performance and scalability** The software is not designed for large-scale structural systems. While the refactoring introduced selective vectorization and systematic use of sparse matrices in key assembly operations, many routines still prioritize readability over maximum efficiency. In principle, further speed-ups would be possible by consistently vectorizing remaining loops, reducing temporary allocations, or using more specialized solvers (e.g. iterative or matrix-free approaches) for very large systems.

However, the intended educational use case typically involves relatively small examples with a limited number of nodes and elements, comparable to problems solved in the context of *Baustatik II*. For this target scope, runtime is not a bottleneck. The code intentionally stores intermediate quantities explicitly (matrices, vectors, intermediate results) to support inspection in the MATLAB workspace, even if this is less memory-efficient than a strictly minimal implementation.

**Generality and extensibility of the solver architecture** While the refactored codebase is more modular than the original implementation, parts of the backend are still tailored to the currently supported modeling scope (2D beam and frame systems with the existing set of element options, loads, springs, prescribed displacements, and influence lines). A more general architecture – for example one that anticipates multiple solver variants, element libraries, or alternative analysis modes – could further reduce friction for future extensions.

Achieving this would likely require introducing additional abstraction layers (e.g. interfaces for solver backends, element types, or load models) and potentially adopting object-oriented patterns such as polymorphism. While such an approach could make it easier to integrate future solvers, it would also increase complexity and reduce accessibility for students with limited programming experience.

**GUI-solver interface and duplicated conversion logic** A key limitation is that the GUI layer and solver layer use different internal representations of the same modeling information. The graphical interface was developed after the solver refactoring, and the original codebase was not designed with App Designer tables and UI-specific data types in mind. As a result, multiple

translation steps are currently required (e.g. `guiToInput`, `uiDataToTable`, and `InputUmwandeln`) to normalise GUI state into the solver-compatible input structure.

This has several consequences:

- the conceptual “distance” between what the user enters and what the solver receives is larger than necessary,
- validation logic is partly duplicated (GUI-level validation for live preview and backend validation prior to computation),
- functions such as `buildPreviewModel` become relatively long because they must construct a consistent intermediate representation from partial UI state.

A more tightly integrated design could reduce these conversions by defining a single canonical data model and deriving both GUI table views and solver input from it. In such a setup, validation could be unified (or at least share a larger common core), and the preview model could be constructed more directly.

**Result presentation and integration into the GUI** The project did not aim to comprehensively redesign the output and plotting layer. While targeted improvements were made (notably more robust scaling and label placement for loads and internal force diagrams), result plots are still displayed in separate MATLAB figure windows rather than being fully embedded into the GUI. A tighter integration of result visualization into the App Designer interface could make the user experience feel more cohesive and would allow additional interactive features (e.g. selecting elements directly in the viewer, toggling result overlays, or exporting figures from within the GUI).

**Testing coverage of output routines** A further shortcoming concerns test coverage. The unit and system tests introduced during the refactoring focus on the DSM core, validation routines, and solver correctness. Output and visualization functions were largely excluded, since they were not expected to change substantially.

During the final development phase, however, improvements to scaling and labeling required touching parts of the internal-force distribution and plotting pipeline. This highlighted that at least the numerical parts of the output chain (e.g. internal-force evaluation, discretization along elements, and result vector assembly) would benefit from unit tests. In future work, it is therefore recommended to add tests for functions such as `schnittkraefte.m` and for any routines that compute the underlying data used for plots. Verifying the numerical arrays that drive the plots would substantially reduce the risk of regressions when improving visualization.

## 8 Future Work and Expansion Potential

The potential for future projects building upon this software was a key consideration throughout the development process. Rather than aiming for a feature-complete solution, the project was designed as a robust and extensible foundation that can be expanded in multiple directions. The long-term vision is to evolve this software into a unified educational framework that combines numerical correctness, didactic transparency, and interactive exploration within a single coherent environment.

There are several natural directions in which the project could be extended. These range from incremental quality-of-life improvements and output refinements, to deeper didactic features and substantially extended analysis capabilities. In addition, further refactoring and architectural refinements could improve generality and reduce complexity in the long term. Many of the ideas outlined below emerged during weekly project meetings but could not be realised within the limited time frame of a single semester. They are therefore collected here to serve as inspiration and a concrete starting point for future student projects.

**Incremental feature extensions** Without fundamentally changing the solver logic, a number of relatively contained additions could improve the completeness and usability of the current version of the software. Examples include:

- support for angled supports,
- support for angled forces,
- explicit unit handling within the GUI,
- symbolic solutions expressed in terms of parameters such as  $EA$ ,  $L$ , and  $F$ ,
- additional distributed load shapes, including user-defined load functions,
- visualization of displacements and nodal deformations as result quantities.

**Output and visualization overhaul** The output and result presentation layer also have room for improvement. While the current implementation is functionally correct, a more integrated and interactive output concept could substantially improve the perceived completeness of the application. Possible improvements include:

- user-controlled scaling of force vectors and diagrams,
- color-coding of forces based on magnitude,
- full integration of result plots into the GUI instead of separate MATLAB figure windows,
- improved presentation of internal force diagrams, including clearer indication of extrema, zero crossings, and numerical values,
- tighter coupling between input fields and the live viewer by making structural elements clickable to highlight the corresponding input rows.

**Refactoring and architectural refinements** Although the refactoring carried out in this project significantly improved structure and maintainability, further architectural refinements are conceivable. In particular:

- aligning the solver input data structures more closely with the data produced by the GUI, thereby reducing the need for multiple conversion and validation layers,
- streamlining the `Model` structure by removing redundant fields and retaining only quantities that serve a numerical or didactic purpose, especially within `Model.Analyse.kond`,
- revisiting the node numbering strategy used for influence line calculations, with a clearer distinction between original nodes and auxiliary nodes introduced to discretise beams.

Addressing these points could reduce code complexity and simplify both backend logic and visualization.

**Didactic extensions** One of the most promising directions for future work lies in expanding the educational functionality of the software. As the tool is primarily intended to support learning and conceptual understanding, a wide range of didactic features could be integrated, for example:

- introduction of a dedicated *learning mode*, potentially integrated into the information pop-up window, which exposes additional explanatory content,
- visualization of local and global stiffness matrices and force vectors, with color-coding that links matrix entries to corresponding structural elements,
- step-by-step solution walkthroughs during computation, including short theoretical explanations of each DSM step,
- completion and expansion of the static condensation feature to explicitly visualize intermediate matrices, reduced systems, and the formulas used to perform static condensation.

**New analysis features** Beyond incremental improvements, the software could also be extended with entirely new analysis capabilities. The range of possibilities is broad and includes, for example:

- load combinations and load envelopes with safety factors,
- extended material definitions, including nonlinear laws, composite materials, or anisotropic behavior,
- cross-section modeling with graphical definition and automatic computation of sectional properties,
- buckling analysis and second-order effects,
- computation and visualization of initial buckling modes,
- thermal strains and thermally induced displacements,

- plastic analysis with visualization of hinge formation and collapse mechanisms,
- dynamic analyses such as natural frequencies, mode shapes, and harmonic response,
- incorporation of strut-and-tie concepts, for example through optimization-based truss representations within beams.

Some of these extensions would require a shift towards a more general finite-element-based formulation or would go beyond the intended scope of *Baustatik II*. Depending on the long-term vision for the software, it may therefore be more appropriate to address such developments in a separate project or a new codebase, rather than integrating them incrementally into the present system.

## 9 Outlook and Conclusion

The aim of this project was to improve the usability and maintainability of an existing DSM-based solver and to extend it into an interactive educational tool for *Baustatik II*. The result is a unified software framework that combines a refactored DSM core with a graphical user interface, supporting both structured computation and exploratory learning.

**Achievements** The DSM implementation was systematically refactored to improve clarity, modularity, and alignment with the stepwise procedure taught in the course. In addition, a layered sanitization and validation concept was introduced, together with unit and system tests that help ensure correctness and reduce the risk of regressions during future development. The graphical user interface enables intuitive system definition, incremental validation, and live visual feedback, transforming the solver from a script-based tool into a usable application for students.

**Educational value** The software supports learning by allowing students to interactively construct structural models and directly observe the effects of modeling decisions on structural behavior. By keeping the computational workflow close to the hand-based DSM method, the tool reinforces concepts introduced in lectures while enabling analyses that go beyond manual calculation. Immediate visual feedback and clear error messages help students identify and correct modeling mistakes early.

**Sustainability and outlook** Rather than representing a finished or feature-complete tool, the software was intentionally developed as a stable starting point for continued work. Its structure allows future projects to focus on specific aspects – such as didactic extensions, visualization concepts, or selected additions to the analysis workflow – without requiring fundamental changes to the solver core. This makes the project suitable not only for student use, but also as a basis for future work.

## 10 References

- (2025) *MATLAB App Designer*, The MathWorks, Inc., <https://www.mathworks.com/products/matlab/app-designer.html>.
- Bundi, T. and D. Ferrari (2018) Interactive educational software for structural analysis. Unpublished.
- Kirupakaran, T. (2023) Dsm-based educational software for structural analysis. Unpublished.
- Roslovets, P. (2025) Icons for matlab app designer and simulink, <https://github.com/roslovets/Icons-for-MATLAB>. Version 1.3.1.
- Salathé, M. and S. Beeler (2020) Educational software for teaching the direct stiffness method. Unpublished.
- Varga, D. (2024) Mbeautifier: Matlab code formatter, <https://github.com/davidvarga/MBeautifier>. Version 1.4.0.

## A User Guide: Graphical User Interface

This appendix provides a practical user guide for creating, analyzing, and exporting structural systems using the MATLAB GUI. The workflow is sequential and follows the tab order of the interface. Users may either start from an empty model or import a predefined example system.

### A.1 Getting Started

**Launching the GUI** Start MATLAB from the project root (e.g. via `startup.m`) and launch the interface by typing: `GUI`

**Importing / Exporting Systems** At any time, a complete system definition can be exported or imported using the corresponding buttons in the lower left:

- **Export:** saves all GUI inputs (including analysis settings) to a MATLAB `.mat` file.
- **Import:** loads a previously exported `.mat` file and populates all tables automatically.

### A.2 Workflow Summary (Checklist)

A typical workflow consists of:

1. Define **Materials** and **Cross-Sections**.
2. Define **Nodes** and **Beams** (including hinges), then assign **Supports** and optional **Springs**.
3. Define **Actions**: nodal loads, member loads, and prescribed displacements.
4. In **Results**, choose output type(s) and press **Calculate**.
5. Inspect plots in the GUI; inspect numerical matrices/vectors in the MATLAB workspace if required.

### A.3 Tab Descriptions

**Material Tab** Define material properties required for the structural model. Each row corresponds to one material and requires:

- Material name,
- Young's modulus  $E$ .

Rows can be added using the plus button below the table or via the table context menu (*Hinzufügen*). The info button above the table opens a context-specific description of the required inputs. Clearing and closing functions are available via the tab controls.



**Cross-Section Tab** Define cross-sections used by beam elements. Each row corresponds to one cross-section and requires:

- Cross-section name,
- Assigned material (selected from a drop-down menu),
- Area,
- Second moment of area.

The material drop-down menu lists all materials defined in the Material tab. If required values are missing or invalid (e.g. negative area or inertia), the GUI blocks progression and displays a descriptive error message.

**Beam Tab** This tab defines the structural geometry and boundary conditions. It contains tables for:

- **Nodes:** define node coordinates. Fully defined nodes appear immediately in the live viewer.
- **Beams:** define connectivity by selecting start node, end node, and cross-section via drop-down menus.
- **Hinges:** define end releases for beams. Available hinge types are:
  - No hinge
  - Normal force hinge
  - Shear force hinge
  - Moment hinge
- **Supports and Springs:** assign boundary conditions and optional springs to nodes.

The live viewer displays node numbers and beam numbers. Beam orientation is indicated with a directional marker. Legend entries can be toggled to show/hide categories and labels.

**Actions Tab** Define external actions acting on the system:

- Nodal loads,
- Member loads (concentrated or distributed),
- Prescribed displacements.

Nodal and member loads are specified separately to reflect DSM modeling decisions. For member loads, locations are defined by a normalized coordinate in  $[0, 1]$ , representing the relative position along the beam length.

**Results Tab** Select which results should be computed and visualized. Available result types include:

- internal forces,
- support reactions,
- influence lines.

The desired output is chosen via a drop-down menu. Depending on the output type, additional fields appear (e.g. response quantity and evaluation location for influence lines).

### Static Condensation (Optional)

For internal forces and support reactions, the GUI offers an option to apply static condensation. When enabled, a table is shown for specifying degrees of freedom to be condensed. The system is then solved for the reduced and full degree-of-freedom set. Condensed system quantities can be inspected in the MATLAB workspace or command window:

```
full(Model.Analyse.kond.K_ff_red)
```

This feature is intended primarily for educational purposes.

## A.4 Validation and Error Messages

The GUI validates input at multiple stages:

- **Row-level validation:** completed rows are checked before they appear in the live viewer.
- **Tab-level validation:** all tables in the current tab are checked when switching to the next tab.
- **Pre-solve validation:** full input is validated when **Calculate** is pressed.

If an issue is detected, the GUI shows an error dialog and blocks progression until the input is corrected.

## A.5 Live Viewer Interaction

The live viewer continuously visualizes the system during model construction. The legend allows toggling visibility of individual object types and labels. The viewer is designed to remain stable under partial input: only complete and valid entities are visualized, and dependent definitions are updated consistently when objects are removed or modified.

## B Example Results from the Software

Figure 10: Example system and influence line results

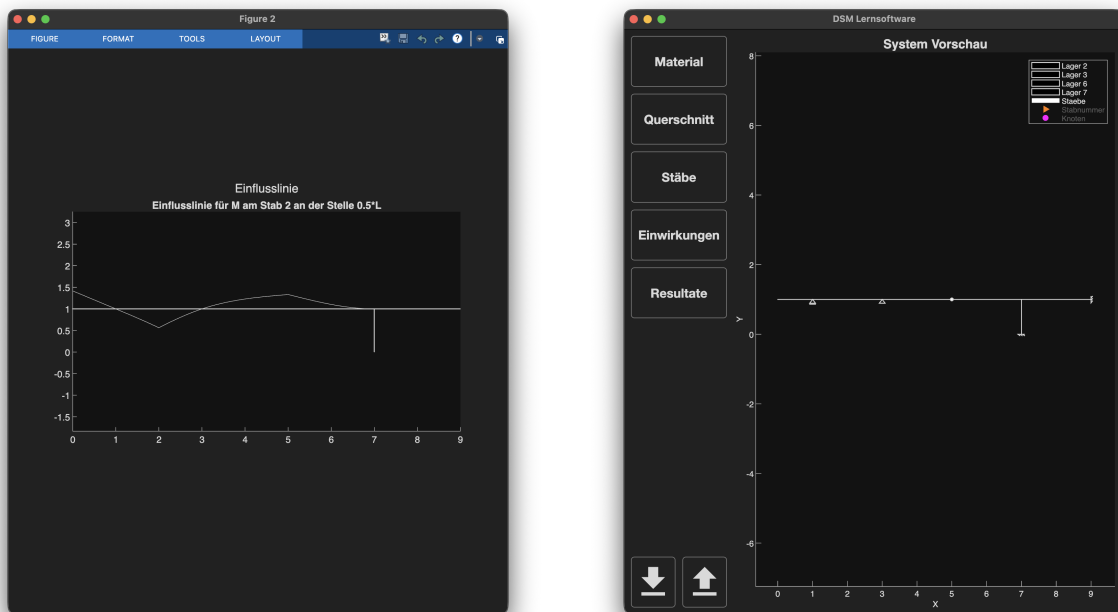


Figure 11: System definition and live visualization in the graphical user interface

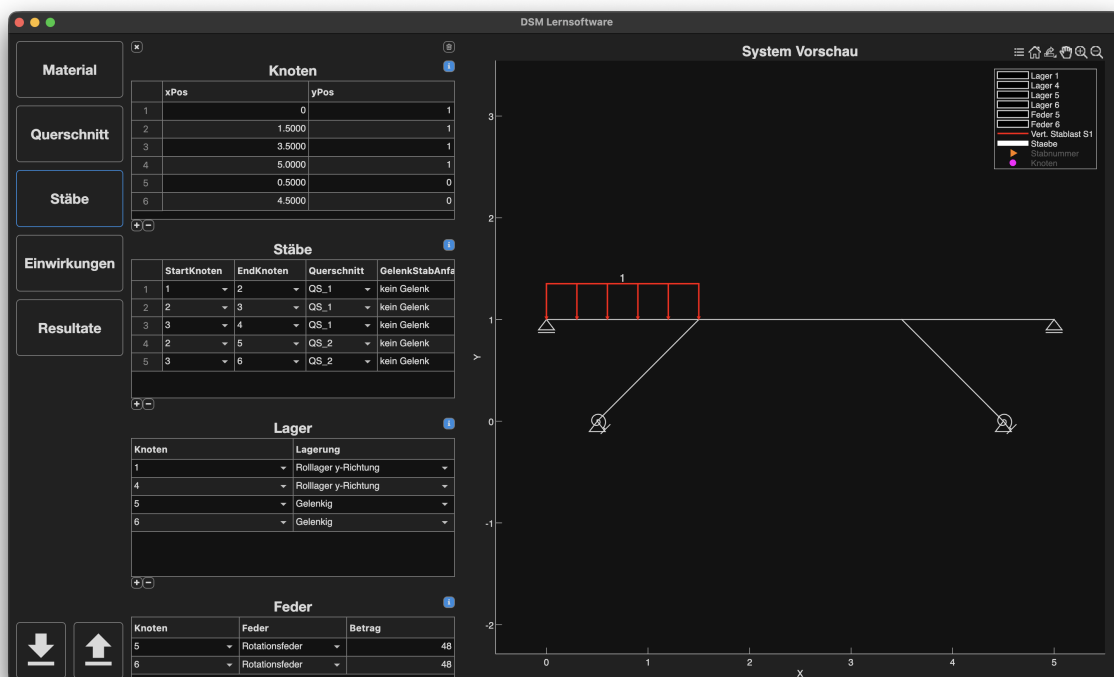


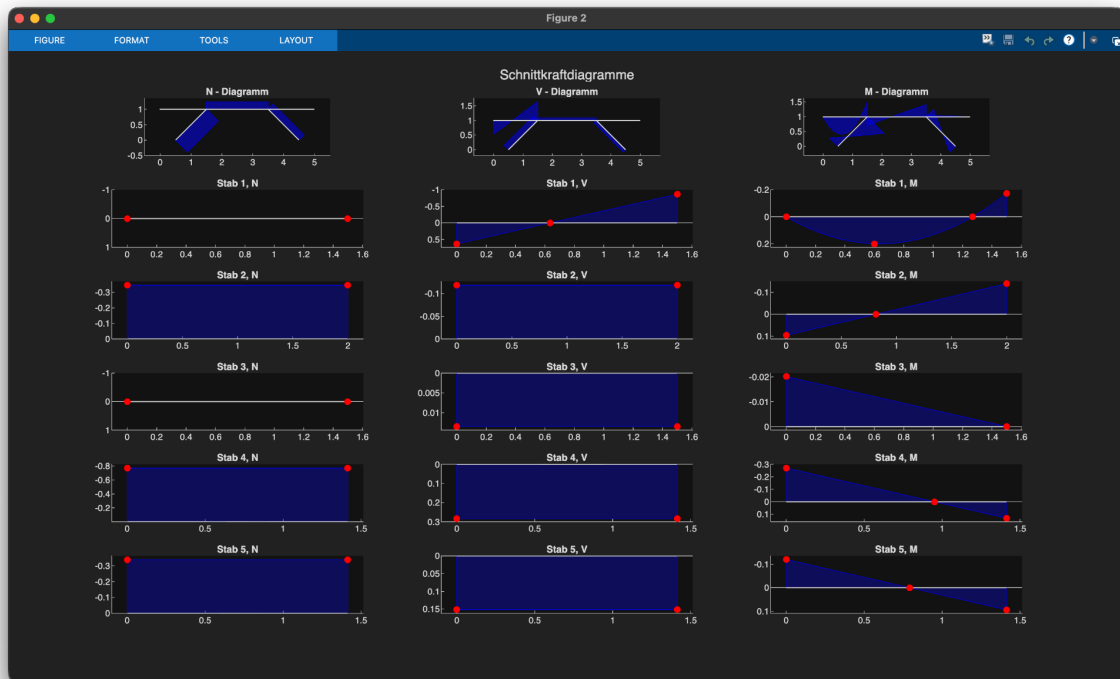
Figure 12: Internal force diagrams (normal force  $N$ , shear force  $V$ , bending moment  $M$ )

Figure 13: Support reactions computed by the DSM solver

